

Vectors and Matrices

Linus Hagemann

Review of Vectors

We covered the basics of vectors in the [Introduction to R slides](#). To recap, vectors are an ordered collection of information. For example

```
1 example_vector <- c(1:10)
2 example_vector
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

As a reminder, `c()` is a function that creates a vector. As you can see above, `c(1:10)` is a vector containing every integer between 1 and 10.

Vector Operations

To recap, vector operations are element-wise. For example

```
1 c(1:10) * c(1:10)
```

```
[1] 1 4 9 16 25 36 49 64 81 100
```

multiplies the 1st element of each vector together (1 times 1), the 2nd element of each (2 times 2), and so on.

Vectors of Different Lengths

If you do vector operations with vectors of different lengths, the elements of the shorter vector are recycled. For example

```
1 c(1:10) * c(1,2)
```

```
[1] 1 4 3 8 5 12 7 16 9 20
```

1 is multiplied by 1, 2 is multiplied by 2, 3 is multiplied by 1, 4 is multiplied by 2, and so on.

Watch out for this—vector recycling is often unintended and a source of bugs.

Vector Types/Classes

Each element of a vector has to be the same kind of thing. R usually “coerces” (i.e. changes) different elements to the same kind inside vectors if you include various kinds inside `c()`. For example

```
1 c(1, 2, 3, "Hello")  
[1] "1"      "2"      "3"      "Hello"
```

As a rule of thumb, R will coerce elements to strings because almost anything can be represented as a string.

Logical Vectors

Vector operations with logical operators such as `&`, `|`, `>`, and `<` return `TRUE` or `FALSE`. For example

```
1 c(1:10) > 5
```

```
[1] FALSE FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE
```

Logical vector operations are used extensively when subsetting dataframes and other kinds of data with base R.

Exercise

The modulo arithmetic operator `%%` returns the remainder after division. For example

```
1 c(2:5)
```

```
[1] 2 3 4 5
```

```
1 c(2:5) %% 2
```

```
[1] 0 1 0 1
```

How can we use this to identify even numbers between 1 and 10?

Hints: We can subset a vector using another vector with `[]`. Recall that `==` checks that an element is equal to another element.

Solution

```
1 c(1:10) %% 2
```

```
[1] 1 0 1 0 1 0 1 0 1 0
```

```
1 c(1:10) %% 2 == 0
```

```
[1] FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE
```

```
1 c(1:10)[c(1:10) %% 2 == 0]
```

```
[1] 2 4 6 8 10
```


Named Vectors

Elements inside a vector can have names. For example

```
1 berlin_pop <- c(  
2   "1980" = 3048759,  
3   "1990" = 3433695,  
4   "2000" = 3382169,  
5   "2010" = 3460725,  
6   "2020" = 3664088  
7 )  
8 berlin_pop
```

1980	1990	2000	2010	2020
3048759	3433695	3382169	3460725	3664088

Exercise

How can we get the population of Berlin in years where the population exceeds 3.4 million?

Solution

```
1 berlin_pop[berlin_pop > 3400000]
```

	1990	2010	2020
	3433695	3460725	3664088

Indexing Named Vectors

You can access the elements of a named vector by their names. For example, we can get Berlin's population in the year 2000 with

```
1 berlin_pop["2000"]
```

```
2000  
3382169
```

You can select multiple elements by name by putting the names in their own vector

```
1 berlin_pop[c("2000", "2010", "2020")]
```

```
2000    2010    2020  
3382169 3460725 3664088
```

Matrices

A matrix is a two-dimensional set of values. You can think of it as a vector of vectors. A matrix is similar to a dataframe in that it has rows and columns.

Indexing Matrices

Rather than accessing elements with one index (`[index]`, where `index` is either a number or name), elements are accessed with a row and column index (`[row_index, column_index]`) as implied by the column and row labels below

```
1 example_matrix
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	1	6	11	16	21
[2,]	2	7	12	17	22
[3,]	3	8	13	18	23
[4,]	4	9	14	19	24
[5,]	5	10	15	20	25

Indexing Matrices

We can get the first row of `example_matrix` with

```
1 example_matrix[1,]
```

```
[1] 1 6 11 16 21
```

The first column with

```
1 example_matrix[,1]
```

```
[1] 1 2 3 4 5
```

Note that indexing a single row or column of a matrix returns a vector.

Indexing Matrices

We can get an individual element from a matrix by including both a row and column index.

```
1 example_matrix[1,1]
```

```
[1] 1
```


Exercise

Get the elements 5, 13, 17, and 23 out of `example_matrix`.

Solution

```
1 example_matrix[5,1]
```

```
[1] 5
```

```
1 example_matrix[3,3]
```

```
[1] 13
```

```
1 example_matrix[2,4]
```

```
[1] 17
```

```
1 example_matrix[3,5]
```

```
[1] 23
```

Subsetting Matrices

We can subset a matrix by including multiple values in the indices.

```
1 example_matrix[1:3,1:3]
```

	[,1]	[,2]	[,3]
[1,]	1	6	11
[2,]	2	7	12
[3,]	3	8	13

As long as the subset includes more than one row or column, R will return a matrix rather than a vector.

Subsetting Matrices

If you do include an index for the rows/columns, R will return all of the rows/columns

```
1 example_matrix[1:2,]
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	1	6	11	16	21
[2,]	2	7	12	17	22

```
1 example_matrix[,1:2]
```

	[,1]	[,2]
[1,]	1	6
[2,]	2	7
[3,]	3	8
[4,]	4	9
[5,]	5	10

Exercise

Get the 9 values in the center of the matrix (7-9, 12-14, and 17-19).

Solution

```
1 example_matrix[2:4,2:4]
```

	[,1]	[,2]	[,3]
[1,]	7	12	17
[2,]	8	13	18
[3,]	9	14	19

Matrix Operations

Matrix operations are very similar to vector operations. Matrix operations with a scalar (a single element) perform the operation on each element of the matrix.

```
1 example_matrix * 5
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	5	30	55	80	105
[2,]	10	35	60	85	110
[3,]	15	40	65	90	115
[4,]	20	45	70	95	120
[5,]	25	50	75	100	125

Matrix Operations

Matrices support the same logical operations as vectors. For example, we can convert `example_matrix` into a matrix of **TRUE** values when the element is even and **FALSE** values when the element is odd with

```
1 example_matrix %% 2 == 0
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	FALSE	TRUE	FALSE	TRUE	FALSE
[2,]	TRUE	FALSE	TRUE	FALSE	TRUE
[3,]	FALSE	TRUE	FALSE	TRUE	FALSE
[4,]	TRUE	FALSE	TRUE	FALSE	TRUE
[5,]	FALSE	TRUE	FALSE	TRUE	FALSE

Subsetting with Logical Matrices

Matrices can be subset by logical matrices just like vectors can be subset with logical vectors. This will return a vector of all of the elements with a row and column index that corresponds to true in the logical matrix. For example, we can use the logical matrix from the previous slide to get a vector of all the even numbers in `example_matrix`.

```
1 example_matrix[example_matrix %% 2 == 0]
```

```
[1]  2  4  6  8 10 12 14 16 18 20 22 24
```

Exercise

Get all the elements in `example_matrix` that are divisible by 3.
Then subset those elements to those with at least 2 digits.

Solution

```
1 divisible_by_3 <- example_matrix[example_matrix %% 3 == 0]  
2 divisible_by_3[divisible_by_3 >= 10]
```

```
[1] 12 15 18 21 24
```

Matrices and Functions

Many functions (e.g. `sum()`, `min()`, `max()`, and `mean()`) work with matrices. These functions will treat the matrix like a vector and operate on all of the elements in the matrix.

```
1 sum(example_matrix)
```

```
[1] 325
```

```
1 min(example_matrix)
```

```
[1] 1
```

```
1 max(example_matrix)
```

```
[1] 25
```

```
1 mean(example_matrix)
```

```
[1] 13
```

Exercise

How much many times larger is the largest odd number in `example_matrix` compared to the smallest even number?

Solution

```
1 largest_odd <- max(example_matrix[example_matrix %% 2 == 1])
2 smallest_even <- min(example_matrix[example_matrix %% 2 == 0])
3 largest_odd / smallest_even
```

```
[1] 12.5
```